

Personal Expense & Splitwise Sync Bridge

Master Project Specification & Architectural Outline

1. Executive Summary & Core Concept

This project solves a persistent gap in personal finance software: tracking a strict personal budget limit (such as \$800 per month) when your credit cards are regularly used for co-mingled purchases (group groceries, shared tabs, and buying items for others) which are reimbursed asynchronously.

Standard budgeting apps fail because they cannot track running social debts. Splitwise fails because it does not manage personal cash flow or link to personal bank limits. This custom system serves as an intelligent bridge between raw bank transactions and the Splitwise API.

Core Ledger Mechanics: Separating Cash Flow from Expenses

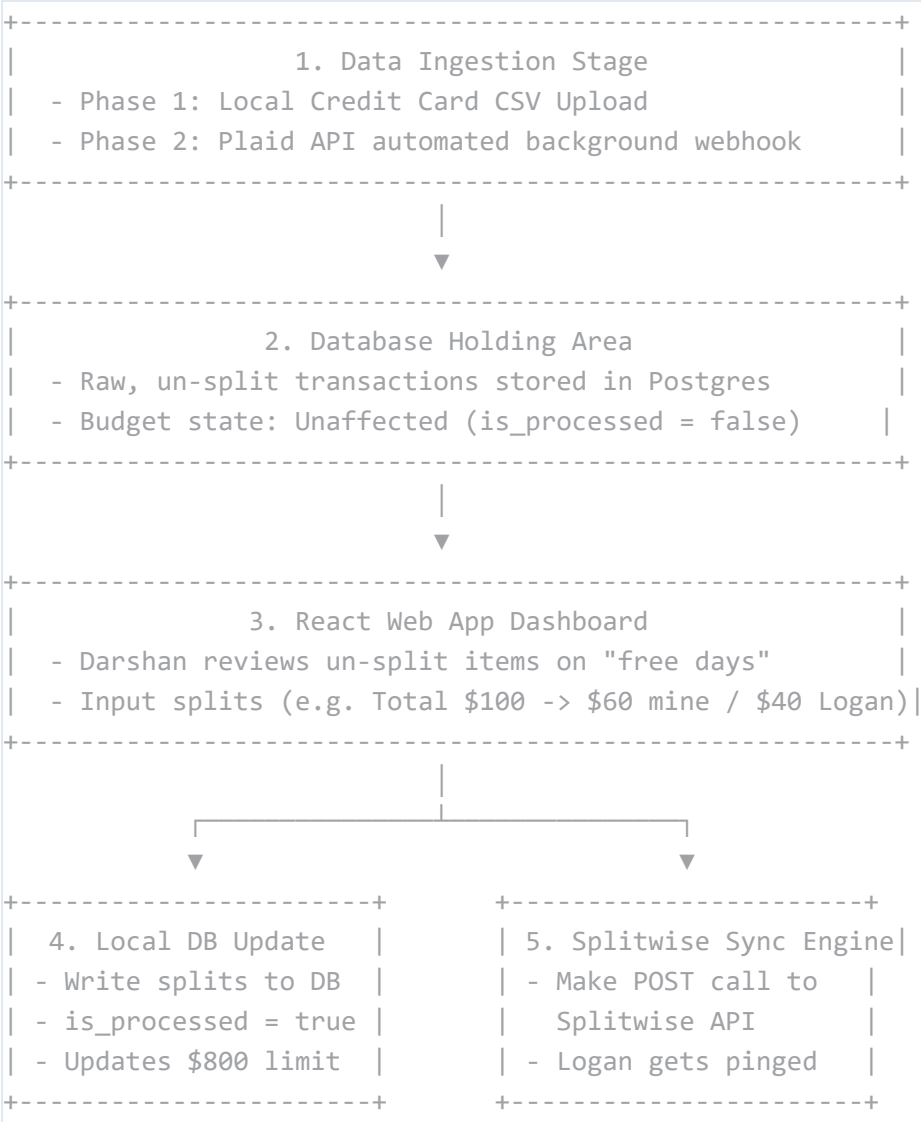
Your application uses a dual-concept ledger system under the hood:

- **Payment Methods (The Cash Flow Source):** Your actual credit card accounts or cash wallet. Purchases are logged here at their 100% face value.
- **Asset Accounts (IOUs - The Receivables):** This represents "Housemates Owe Me" or "Logan Owes Me." Money assigned here is an asset, not an expense.
- **Expense Accounts (The Consumption - Your \$800 Limit):** This represents what you actually consumed (e.g., your portion of groceries, your personal coffee). **Only these lines subtract from your \$800 monthly limit.**

When you pay \$100 for groceries at Walmart, the full \$100 is written to your Credit Card ledger. When you split the transaction on your "free days," \$60 is assigned to your personal Grocery Expense (hitting your budget) and \$40 is assigned to the "Logan Owe Me" Asset Account. When Logan reimburses you, the \$40 shifts from "Logan Owe Me" to your cash/checking account. Your \$800 personal limit is never artificially inflated.

2. Technical System Architecture

Below is the conceptual flow of the hybrid architecture. It starts with your physical credit card purchase and ends with a sync to both your local budget dashboard and the Splitwise cloud registry.



3. Database Schema Design (PostgreSQL)

This relational database structure supports multi-party splitting, tracking unprocessed imports, and linking to your outbound Splitwise logs.

```
-- Core Accounts representation (Credit cards, cash, and IOU placeholders)
CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL, -- e.g., 'TD Visa Infinite', 'Cash Wallet', 'Roommate
Receivables'
  type VARCHAR(50) NOT NULL, -- e.g., 'credit_card', 'cash', 'asset_iou'
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Master table for raw bank transactions
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_date DATE NOT NULL,
  merchant VARCHAR(255) NOT NULL,
  original_amount DECIMAL(10, 2) NOT NULL, -- The 100% cost as seen on the card
  currency VARCHAR(10) DEFAULT 'CAD',
  payment_method_id UUID REFERENCES accounts(id),
  is_processed BOOLEAN DEFAULT FALSE, -- Checked once you split the item
  external_bank_id VARCHAR(255), -- Unique ID from Plaid or CSV to prevent
duplicates
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Splits table (A single transaction has 1 or many split rows)
CREATE TABLE transaction_splits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID REFERENCES transactions(id) ON DELETE CASCADE,
  split_type VARCHAR(50) NOT NULL, -- 'personal_expense', 'roommate_iou'
  assigned_to_user VARCHAR(100) NOT NULL, -- 'Darshan', 'Logan', 'Housemates Group'
  amount DECIMAL(10, 2) NOT NULL, -- The split portion (e.g., $60.00 or $40.00)
  splitwise_expense_id VARCHAR(100), -- Tracks corresponding ID in Splitwise
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Optional: Cache Splitwise user/group configurations locally to accelerate your UI
CREATE TABLE splitwise_entities (
  id VARCHAR(100) PRIMARY KEY, -- Splitwise internal User/Group ID
  name VARCHAR(255) NOT NULL,
  type VARCHAR(50) NOT NULL -- 'friend' or 'group'
);
```

4. Azure Boards Project Backlog Hierarchy

This backlog is structured for direct import or manual entry into Azure DevOps, following the Epic to Feature to User Story to Task framework.

Epic 1: Local Ledger Engine & Database (The Foundation)

Target: Sprint 1

Feature 1.1: Database Infrastructure & Schema

User Story: As a developer, I want a relational database schema to hold transaction records, splits, and user profiles.

- Task 1.1.1: Spin up a free serverless PostgreSQL instance on Neon or Railway.
- Task 1.1.2: Execute table migration scripts for schemas.
- Task 1.1.3: Seed the database with typical test records (Costco, Walmart, Cafe).

Feature 1.2: CSV Data Ingestion Engine

User Story: As Darshan, I want to parse bank credit card CSV exports so that my transactions automatically enter the system inbox.

- Task 1.2.1: Create a Node.js Express template project with TypeScript support.
- Task 1.2.2: Write a modular CSV parsing service using csv-parser.
- Task 1.2.3: Write safe INSERT logic checking a composite key (date, amount, merchant) to ignore duplicate imports.
- Task 1.2.4: Create a secure route POST /api/transactions/upload.

Feature 1.3: Core Budget Calculation Math

User Story: As Darshan, I want the system to isolate my actual personal expenses so that I can see how close I am to my \$800 monthly limit.

- Task 1.3.1: Write a SQL/ORM query summing up personal splits for the current calendar month.
- Task 1.3.2: Create a dashboard status controller GET /api/budget/status.

Epic 2: Splitwise Integration (The Automated Outbound Layer)

Target: Sprint 2

Feature 2.1: Splitwise API Client Setup

User Story: As a developer, I want to connect securely to the Splitwise API so my backend can push transactions.

- Task 2.1.1: Register a new custom application in the Splitwise Developer Portal.
- Task 2.1.2: Save API Consumer Keys and secrets in a local configuration file (.env).
- Task 2.1.3: Develop a secure authentication handler / client instance inside the application.

Feature 2.2: Automated Expense Splitting Engine

User Story: As Darshan, I want to assign a transaction split to a friend so that Splitwise automatically creates the IOU debt entry for them.

- Task 2.2.1: Design service function mapping transaction_splits to the Splitwise request schema.
- Task 2.2.2: Implement outbound error handling to intercept network issues or failed API requests.
- Task 2.2.3: Write database update logic stamping splitwise_expense_id onto local database records.

Epic 3: Unified User Interface (The Control Panel)

Target: Sprint 3

Feature 3.1: The Unprocessed Inbox View

User Story: As Darshan, I want a dedicated feed of unallocated transactions so I can quickly see what needs to be reviewed.

- Task 3.1.1: Create a clean React dashboard page displaying an "Inbox".
- Task 3.1.2: Fetch all transactions where `is_processed = false` from the backend.
- Task 3.1.3: Build a scrollable list card components showing raw transactions.

Feature 3.2: Interactive Split Selector

User Story: As Darshan, I want to click an item and assign amounts to myself and roommates without doing mental math.

- Task 3.2.1: Build an interactive Modal component triggered by selecting inbox rows.
- Task 3.2.2: Add preset shortcut buttons (50/50, 100% Personal, Custom amount inputs).
- Task 3.2.3: Build live state calculator showing remaining unassigned values.
- Task 3.2.4: Connect confirmation submit handlers to `POST /api/transactions/:id/split`.

Feature 3.3: Visual Budget Progress Dashboard

User Story: As Darshan, I want a clear visual progress bar tracking my real \$800 limit so I know instantly if I need to slow down spending.

- Task 3.3.1: Build a modern progress bar widget utilizing custom visual alerts for budget thresholds.
- Task 3.3.2: Display critical stats: Real Spending, Outgoing IOUs Pending, Days Remaining.

Epic 4: Advanced Frictionless Ingestion (Plaid & Voice Integration)

Target: Sprint 4

Feature 4.1: Live Bank Sync with Plaid

User Story: As Darshan, I want my credit card transactions to sync automatically to my inbox so I do not have to download statements.

- Task 4.1.1: Implement the frontend Plaid Link UI component in React.
- Task 4.1.2: Construct the webhook handler `POST /api/plaid/webhook` on Express.
- Task 4.1.3: Use Plaid's `/transactions/sync` endpoint to populate your local database.

Feature 4.2: On-The-Go AI Voice Processing

User Story: As Darshan, I want to dictate an expense split on my phone so it automatically processes without using a computer.

- Task 4.2.1: Integrate OpenAI's Whisper API to transcribe audio files.
- Task 4.2.2: Construct system prompt parsing sentences into structured JSON split objects.
- Task 4.2.3: Pipe output directly to endpoint to bypass inbox manual step.

5. Phase 1 Execution Details (Getting Started with Sprint 1)

As planned, we are going to start with **Sprint 1: The Core Database and CSV Parser**. This allows us to get the system fully operational locally before writing any complex third-party authentication code.

Recommended Steps to Initialize Sprint 1:

1. **Initialize PostgreSQL Database:** Spin up a free instance using Neon or Railway. Store the database connection string securely in a local `.env` file.
2. **Boilerplate Set Up:** Initialize a Node.js Express server with TypeScript.
3. **Create Tables:** Run the SQL creation script from Section 3 of this document to set up your tables.
4. **Write the CSV Parser:** Write a simple utility using `csv-parser` that takes a standard CSV download from your credit card company and outputs a clean JSON array mapping the Date, Merchant Name, and Amount.
5. **Write Ingest Database Logic:** Create a controller function that reads this JSON array and inserts each row into the `transactions` table (setting `is_processed = false` by default). Make sure you check for duplicates so you can upload the same CSV file repeatedly without messing up your ledger database.

6. Cloud Deployment Targets

To keep your project completely free while ensuring a premium developer experience, deploy using the following cloud stack:

- DB Database (Neon):** Serverless Postgres with a highly capable free tier that scales database compute engines to zero when idle.
- API Backend Server (Railway):** Provides automated deployments directly from your GitHub commits, isolated node runtimes, and easy environment variable setup.
- WEB Frontend (Vercel):** Fast static global CDN hosting for your React app dashboard. It auto-deploys on every git commit.